



rPPG DEEPFAKE DETECTION SYSTEM

Team Execution Playbook

Biometric Physics vs Synthetic Media

A DARPA-Level Cybersecurity Defense

4-Person Team | 4–5 Hours | Python + OpenCV + AI

STRICTLY NO CODE IN THIS DOCUMENT — PROCEDURE, ARCHITECTURE & CHECKLISTS ONLY

SECTION 1: MISSION BRIEFING

The Problem, The Science, The Objective

1.1 The Threat Landscape

Deepfakes are AI-synthesized videos indistinguishable from real footage by the human eye. They are weaponized in:

- National security breaches — fabricated video of world leaders issuing false commands
- Election interference — synthetic videos of political candidates making false statements
- Identity fraud — bypassing biometric video-verification systems used by banks and governments
- Corporate espionage — faking executive instructions (CEO fraud) to authorize wire transfers

△ *The critical insight: current detection tools analyze pixels, lighting artifacts, and GAN fingerprints. Generative models are outpacing these defenses every 6–12 months. We need a detection paradigm that is physics-invariant — one that cannot be trained away.*

1.2 The Biological Signal: rPPG

Remote Photoplethysmography (rPPG) is a technique to measure heart rate from subtle, invisible color changes on the skin surface. Every heartbeat causes a micro-surge of oxygenated blood through facial capillaries. This changes the light absorption/reflection properties of the skin by 0.1–1% in amplitude — imperceptible to the human eye but detectable by a camera.

Why rPPG Works on Humans	Why Deepfakes Fail rPPG
Skin reflects red/green/blue light	Deepfake has no real blood flow
Oxygenated blood absorbs more green light	Rendering engines have no cardiac model
Each heartbeat → tiny green channel dip	rPPG signal will be flat (no heartbeat)
Camera captures this at 30fps	Or synthetic = mathematically perfect loop
Signal frequency = heart rate (0.7–3Hz)	No biological noise = no human
Peak in green channel is strongest signal	Flag immediately as THREAT

1.3 Our System's Detection Logic

The system applies three detection layers simultaneously:

1. Extract rPPG signal from facial skin by averaging pixel values in the green channel across the Region of Interest (ROI) over time to form a time-series signal.
2. Analyze signal quality — a real human will show a noisy, biological heartbeat oscillation (0.7–3 Hz). A deepfake shows either no signal (flat line) or a perfect sinusoidal loop (mathematical artifact).

3. Flag as threat if: (a) no pulse detected for > 3 seconds, (b) signal SNR is above a threshold indicating synthetic purity, (c) autocorrelation detects an exact repeating loop pattern.

💡 *Scientific basis: This approach is grounded in peer-reviewed research (e.g., de Haan & Jeanne 2013 CHROM method, Wang et al. 2017 POS method). The signal is real, measurable, and currently impossible for generative models to fake without access to the cardiac simulation pipeline.*

SECTION 2: PROJECT FOLDER STRUCTURE

Shared Directory Layout — Set Up Before Coding Begins

2.1 Master Directory Tree

Create the following structure on the shared drive / Git repository before any work begins. Every team member must clone this repo and never create files outside their designated modules.

Path / File	Purpose & Owner
 deepfake_rppg/	ROOT — all work lives here
 main.py	Entry point — Person A wires modules together
 requirements.txt	Shared dependencies — Person A creates, all update
 README.md	Run instructions — Person D writes at end
 .gitignore	Exclude __pycache__, .env, large video files
 config.py	All constants (thresholds, FPS, ROI params) — Person A
	
 modules/	Core logic — one subfolder per person
 face_detection/	 Person A owns this module entirely
 face_detector.py	Haar/DNN face detection + ROI extraction
 roi_extractor.py	Forehead/cheek sub-region isolation
 __init__.py	Module exports
	
 rppg_engine/	 Person B owns this module entirely
 signal_extractor.py	Green channel averaging over ROI frames
 preprocessor.py	Detrending, normalization, windowing
 rppg_algorithms.py	CHROM / POS / GREEN channel methods
 __init__.py	Module exports
	

 threat_analyzer/	 Person C owns this module entirely
 <code>fft_analyzer.py</code>	<i>FFT + bandpass filtering for heartbeat freq</i>
 <code>pattern_detector.py</code>	<i>Loop detection via autocorrelation</i>
 <code>threat_scorer.py</code>	<i>Aggregates signals → REAL / FAKE / UNCERTAIN</i>
 <code>__init__.py</code>	<i>Module exports</i>
	
 ui/	 Person D owns this module entirely
 <code>display.py</code>	<i>OpenCV overlay: BPM, confidence, THREAT flag</i>
 <code>plotter.py</code>	<i>Real-time matplotlib/cv2 rPPG signal graph</i>
 <code>logger.py</code>	<i>CSV/JSON result logging with timestamps</i>
 <code>__init__.py</code>	<i>Module exports</i>
	
 tests/	Unit tests — each person writes tests for their module
 <code>test_face_detection.py</code>	<i>Person A</i>
 <code>test_rppg_engine.py</code>	<i>Person B</i>
 <code>test_threat_analyzer.py</code>	<i>Person C</i>
 <code>test_ui.py</code>	<i>Person D (optional — visual tests are hard)</i>
	
 data/	Test videos and sample frames for offline testing
 test_real/	Short clips of real faces (webcam recordings)
 test_fake/	Deepfake sample clips (FaceSwap/DeepFaceLab output)
 sample_frames/	Individual .jpg frames for unit testing
	
 docs/	Architecture diagrams, final presentation slides
 <code>architecture.png</code>	<i>System diagram (Person D draws in Draw.io)</i>
 <code>demo_script.md</code>	<i>What to say during the demo — Person D</i>

2.2 Git Workflow Rules (MANDATORY for all 4 members)

 ***Violating these rules will cause merge conflicts that can destroy 2+ hours of work. Follow them without exception.***

4. At the start: ONE person (Person A) creates the repo, pushes the folder skeleton with empty `__init__.py` files in every module, then all others clone it.
5. Each person works ONLY in their designated module folder. Never edit another person's files without verbal agreement.
6. Commit message format: [MODULE] Short description — example: [RPPG] Add CHROM algorithm implementation
7. Pull before every push: `git pull origin main` → resolve any conflicts → then `git push`
8. Branch strategy: main branch only for this sprint. No feature branches. Keep it simple under time pressure.
9. `config.py` is SHARED — if you need to add a constant, announce it in the team chat BEFORE editing to avoid conflicts.
10. Never commit large video files to Git. Put them in `/data/` and add `/data/` to `.gitignore`.

SECTION 3: TEAM ROLES & RESPONSIBILITIES

Who Owns What — Complete Breakdown

Each role is designed so work can proceed in PARALLEL from Hour 1. Dependencies are explicitly called out. No one should be blocked waiting for another person — use placeholder/stub functions where needed.

PERSON A — System Architect & Face Detection *Team Lead / Integration Owner*

- Creates the Git repository, pushes the full skeleton folder structure within the first 15 minutes
- Writes config.py with all shared constants: FPS=30, BUFFER_SIZE=300, ROI_PADDING=10, BANDPASS_LOW=0.7, BANDPASS_HIGH=3.0, THREAT_THRESHOLD=0.85
- Writes main.py that imports all four modules and runs the main video capture loop
- Builds modules/face_detection/face_detector.py using OpenCV's Haar cascade (haarcascade_frontalface_default.xml) or DNN face detector
- Builds modules/face_detection/roi_extractor.py to extract forehead + cheek regions as sub-rectangles of the detected face bounding box
- Defines the DataPacket (dictionary or simple class) that gets passed between modules: {frame, face_bbox, roi_pixels, timestamp}
- Coordinates with all three other team members on interface contracts (what inputs/outputs each module expects)
- Handles final integration in Hour 4: wires Person B, C, D modules into main.py
- Runs the full end-to-end test and coordinates bug fixes

PERSON B — rPPG Signal Engine *Signal Processing Specialist*

- Takes Person A's ROI pixel arrays as input (stub with random pixel arrays if A is not ready)
- Builds modules/rppg_engine/signal_extractor.py: for each frame, average the green channel pixel values within the ROI bounding box to produce a single float value
- Builds modules/rppg_engine/preprocessor.py: implement a circular buffer of BUFFER_SIZE frames, apply linear detrending to remove lighting drift, normalize the signal to zero mean
- Builds modules/rppg_engine/rppg_algorithms.py: implement at minimum the simple GREEN channel method; optionally implement CHROM (de Haan) for better accuracy
- The output of this module is a 1D numpy array (the rPPG time series signal) plus an estimated BPM float (even if approximate)
- Consult scipy.signal for bandpass filtering and detrending — these are the core tools
- Write a standalone test function that can run with a synthetic sine wave input to verify the pipeline works before integration
- Communicate with Person C: define exactly what the rPPG signal array looks like (length, dtype, value range) so C can write the analyzer

PERSON C — Threat Analysis Engine *Algorithm / AI Logic Specialist*

- Takes Person B's rPPG signal array as input (stub with known fake/real signals if B is not ready)
- Builds modules/threat_analyzer/fft_analyzer.py: apply FFT (numpy.fft.rfft) to the signal, apply bandpass filter for 0.7–3.0 Hz (42–180 BPM), find the dominant frequency peak. A real human has a clear peak; a flat/noisy signal or an unnaturally sharp peak indicates synthetic origin
- Builds modules/threat_analyzer/pattern_detector.py: compute the normalized autocorrelation of the signal. If autocorrelation shows perfect periodicity (coefficient > 0.95 at lag intervals), the signal is a mathematical loop — flag as deepfake
- Builds modules/threat_analyzer/threat_scorer.py: aggregates all detection signals into a final verdict: REAL, UNCERTAIN, or THREAT. Uses weighted scoring: no-pulse (40%), loop detection (40%), signal-purity SNR (20%)
- Define clear thresholds in config.py — discuss with Person A before committing
- Create a set of synthetic test signals (real-like: 1.2 Hz sine + noise; fake-like: perfect 1.0 Hz sine; flat: zeros) to validate all three detectors independently
- Output structure: {verdict: "THREAT" | "REAL" | "UNCERTAIN", confidence: 0.0–1.0, bpm: float, loop_detected: bool, pulse_present: bool}

PERSON D — UI / Visualization / Demo Lead *Frontend & Presentation Specialist*

- Takes Person C's verdict dictionary as input (stub with hardcoded verdicts if C is not ready)
- Builds modules/ui/display.py: OpenCV overlay on the live video frame — draw face bounding box (green=REAL, red=THREAT, yellow=UNCERTAIN), display BPM text, display verdict text in large bold font, draw confidence percentage bar
- Builds modules/ui/plotter.py: display a live scrolling graph of the rPPG signal alongside the video using matplotlib with animation, OR embed a small graph as an OpenCV image overlay (matplotlib.backends.backend_agg)
- Builds modules/ui/logger.py: write results to a CSV file (timestamp, verdict, BPM, confidence, loop_detected) for post-demo analysis
- Prepares all test data in /data/: at minimum 2 real face clips and 2 deepfake clips (can source from publicly available deepfake datasets like FaceForensics++)
- Writes docs/demo_script.md — the narrative for the presentation: what to show, in what order, what to say at each step
- Draws the system architecture diagram and saves as docs/architecture.png (use draw.io, free online)
- Writes the final README.md with exact setup and run commands
- Manages the live demo during the presentation: runs the system, explains what the audience is seeing in real time

SECTION 4: MASTER TIMELINE

Hour-by-Hour Execution Plan for All 4 Members

 Clock starts the moment all four team members are at their machines with the repo cloned. Use this timeline as an absolute rule — if a task is not done by its deadline, cut scope, not quality.

Phase	Time	Focus	Milestone
PHASE 0	0:00–0:15	Environment setup — ALL HANDS together	Everyone running, repo live
PHASE 1	0:15–1:30	Parallel module development — Build your module with stubs	Each module runs standalone
PHASE 2	1:30–2:45	Deep implementation — Real algorithms, no more stubs	Real rPPG signal on console
PHASE 3	2:45–3:30	Integration — Person A wires all modules in main.py	Full pipeline on live webcam
PHASE 4	3:30–4:15	Testing & tuning — Validate on real vs fake video	System detects deepfakes correctly
PHASE 5	4:15–4:45	Demo prep — Polish UI, rehearse, finalize README	Demo-ready, README done
BUFFER	4:45–5:00	Emergency fixes only — no new features	Ship it

4.1 Phase 0 (0:00 – 0:15): Environment Setup [ALL 4 MEMBERS TOGETHER]

Do this as a team. Do not split up. Every minute here saves 10 minutes of debugging later.

- Person A creates a new GitHub/GitLab repository (private) named `deepfake_rppg`
- Person A pushes the full directory skeleton with empty `__init__.py`, placeholder `.py` files, and the initial `requirements.txt`
- All three others clone the repository: `git clone [URL]`
- ALL FOUR run: `pip install -r requirements.txt` (required packages: `opencv-python`, `numpy`, `scipy`, `matplotlib`, `imutils`)
- ALL FOUR verify webcam works: `python -c "import cv2; cap = cv2.VideoCapture(0); print(cap.read()[0])"` → must print `True`
- Person A writes and pushes `config.py` with all shared constants immediately
- Confirm shared communication channel (WhatsApp/Slack/Discord) is active for all 4

If any member's webcam does not work: use a pre-recorded video as input (`cv2.VideoCapture("path/to/video.mp4")`) — update `main.py` accordingly.

4.2 Phase 1 (0:15 – 1:30): Parallel Module Development

Everyone works on their own module simultaneously. Use STUBS for dependencies you haven't received yet.

Person A — Face Detection Module

- Download haarcascade_frontalface_default.xml from OpenCV GitHub (or use cv2.data.haarcascades path)
- Implement face detection: load cascade, detect faces in grayscale frame, return the largest face bounding box
- Implement ROI extraction: given a face bounding box, extract the forehead region (top 30% of face, center 60% width) and cheeks (left and right thirds, middle 40% height)
- Write a test: load a JPEG image with a face, confirm bounding box is returned as [x, y, w, h]
- Define the DataPacket dictionary schema: write it as a comment block in __init__.py and share it with the team

Person B — rPPG Engine (use stub: random 480x640x3 numpy arrays as mock frames)

- Build signal_extractor.py: function takes a list of ROI pixel arrays (one per frame), extracts mean green channel value per frame, returns a 1D numpy float array of length = num_frames
- Build preprocessor.py: function takes the raw signal array, applies scipy.signal.detrend, normalizes to [-1, 1] range, returns processed array
- Start rppg_algorithms.py: implement the GREEN method (simplest — just the preprocessed green channel signal)
- Verify with synthetic data: create a perfect 1.2 Hz sine wave, pass it through your pipeline, confirm it survives detrending/normalization intact

Person C — Threat Analyzer (use stub: B's output = synthetic sine array)

- Build fft_analyzer.py: apply numpy.fft.rfft to the signal, compute frequency bins with numpy.fft.rfftfreq, find peak frequency in [0.7, 3.0] Hz range, compute SNR as peak power / mean noise power
- Build pattern_detector.py: compute numpy.correlate(signal, signal, mode='full') normalized, check if peaks repeat at regular intervals with coefficient > 0.92
- Start threat_scorer.py: hardcode a skeleton function that receives the three metrics (pulse_present, loop_score, snr_score) and prints a verdict

Person D — UI Module (use stub: hardcoded verdict dict from C's schema)

- Build display.py: function takes an OpenCV frame + verdict dict, draws a colored rectangle around the face (red=THREAT, green=REAL, yellow=UNCERTAIN), puts text overlays for BPM, confidence, and verdict
- Build logger.py: function opens a CSV file (results.csv) in append mode and writes one row: timestamp, verdict, bpm, confidence
- Start sourcing test videos: download 2 deepfake clips (FaceForensics++ YouTube samples or any public deepfake) and 2 real face videos. Put in /data/ folders.

4.3 Phase 2 (1:30 – 2:45): Deep Implementation

By now everyone should have a working stub. Now implement the real algorithms.

Person A

- Test face detector on multiple images — tune the scaleFactor and minNeighbors parameters so it doesn't miss faces or false-detect
- Implement a face tracking fallback: if face is lost for 1 frame, reuse last known bounding box (prevents signal interruption)
- Begin drafting main.py: set up the cv2.VideoCapture loop, call face_detector, call roi_extractor, accumulate frames into a rolling buffer list

Person B

- Implement the CHROM algorithm: requires computing normalized RGB channels, then applying the formula $[3R - 2G]$ and $[1.5R + G - 1.5B]$, then combining. Reference: de Haan & Jeanne 2013.
- Alternatively implement POS (Plane-Orthogonal-to-Skin): compute mean RGB, normalize, apply $[-\text{skin_vec}_1, \text{skin_vec}_2]$ projection. Reference: Wang et al. 2017.
- Even if CHROM/POS don't fully work, ensure GREEN method is solid — it will produce detectable results for demo purposes
- Add a 5-second warm-up period: signal is marked as CALIBRATING for first 150 frames to let the buffer fill

Person C

- Finalize all three detection metrics into `threat_scorer.py` with tunable weights from `config.py`
- Test on Person B's synthetic signals: (a) 1.2 Hz + noise → should score REAL, (b) perfect 1.0 Hz → should score THREAT, (c) zeros → should score THREAT
- Add time-averaging: keep a rolling window of last 5 verdicts, return the majority vote to prevent flickering between REAL/THREAT

Person D

- Build `plotter.py`: create a small 400x150 matplotlib figure, plot the last 150 values of the rPPG signal as a scrolling line graph, convert to OpenCV image using `fig.canvas.tostring_rgb()`, overlay on bottom of main frame
- Enhance `display.py`: add a circular BPM gauge or animated pulse indicator. The more visual, the better for the demo.
- Pre-record a short webcam video of your own face for testing (10–15 seconds, good lighting). Place in `/data/test_real/`

4.4 Phase 3 (2:45 – 3:30): Integration [Led by Person A]



This is the highest-risk phase. Stay calm. The goal is to get SOMETHING running end-to-end, even if imperfect.

- Person A writes the final `main.py` loop: (1) capture frame → (2) detect face → (3) extract ROI → (4) accumulate buffer → (5) extract rPPG signal → (6) analyze threat → (7) display overlay → (8) log result
- Persons B, C, D: be available in the chat to answer questions about your module's API immediately
- First integration test: run on YOUR OWN FACE. Confirm face is detected, ROI is highlighted, rPPG signal appears on the graph, BPM is displayed (even if wrong).
- Second integration test: run on a test video file (`cv2.VideoCapture("data/test_real/your_face.mp4")`)
- Fix integration bugs in order of severity: crash > wrong output > visual glitch
- Person D: confirm the display overlay does not cause frame drops (target >15 FPS minimum)

4.5 Phase 4 (3:30 – 4:15): Testing & Threshold Tuning

- Test 1 — Real face (live webcam): system should show GREEN box, BPM in 55–90 range, verdict REAL
- Test 2 — Real video clip: run `/data/test_real/ clips`, confirm consistent REAL verdict
- Test 3 — Deepfake video clip: run `/data/test_fake/ clips`, confirm THREAT verdict
- If deepfake is NOT flagged: decrease `THREAT_THRESHOLD` in `config.py` and/or lower the loop detection coefficient. Document what works.
- If real face is being flagged as THREAT: increase `THREAT_THRESHOLD`, increase bandpass filter BPM range, ensure warm-up period is working
- Person C tunes thresholds; Person A re-runs tests; Person B adjusts signal processing if signal quality is poor

- Person D records a short screen-capture video of the system detecting both real and fake — this is your demo insurance

4.6 Phase 5 (4:15 – 4:45): Demo Preparation

- Person D writes README.md: Prerequisites, Installation (pip install -r requirements.txt), Run Command (python main.py), Controls (Q to quit), Interpretation guide (RED = THREAT)
- Person D finalizes docs/demo_script.md: 3-minute presentation narrative
- All members: do a final git pull + git push to confirm clean shared state
- Practice the demo: one full run showing REAL face, then deepfake — time it to under 2 minutes
- Prepare verbal answers for likely judge questions (see Section 7)

SECTION 5: MODULE INTERFACE CONTRACTS

What Each Module Receives and Returns — The API Agreement

 These contracts must be agreed upon and written into `__init__.py` comment blocks *BEFORE* coding begins. This is the most important alignment document for parallel development.

5.1 Face Detection Module Output (Person A → Person B + D)

Key	Type	Description
<code>frame</code>	<code>np.ndarray (H×W×3)</code>	Original BGR frame from cv2
<code>face_detected</code>	<code>bool</code>	True if a face was found this frame
<code>face_bbox</code>	<code>tuple (x, y, w, h)</code>	Bounding box of largest detected face in pixels
<code>roi_forehead</code>	<code>np.ndarray (h×w×3)</code>	Cropped BGR sub-image of forehead region
<code>roi_cheeks</code>	<code>np.ndarray (h×w×3)</code>	Cropped BGR sub-image of combined cheek region
<code>timestamp</code>	<code>float</code>	<code>time.time()</code> value at frame capture

5.2 rPPG Engine Output (Person B → Person C + D)

Key	Type	Description
<code>rppg_signal</code>	<code>np.ndarray (N,)</code>	Processed rPPG time series, N = buffer length
<code>estimated_bpm</code>	<code>float</code>	Dominant heart rate frequency × 60
<code>is_calibrating</code>	<code>bool</code>	True if buffer not yet full (< 5 seconds of data)
<code>signal_quality</code>	<code>float [0-1]</code>	Confidence score based on SNR of signal
<code>method_used</code>	<code>str</code>	"GREEN", "CHROM", or "POS"

5.3 Threat Analyzer Output (Person C → Person D + main.py)

Key	Type	Description
<code>verdict</code>	<code>str</code>	"REAL", "THREAT", or "UNCERTAIN"
<code>confidence</code>	<code>float [0.0-1.0]</code>	How certain the system is of this verdict
<code>bpm</code>	<code>float</code>	Estimated BPM (0.0 if no pulse detected)
<code>pulse_present</code>	<code>bool</code>	True if a heartbeat frequency was found

loop_detected	<code>bool</code>	True if autocorrelation reveals synthetic repetition
snr_score	<code>float</code>	Signal-to-noise ratio metric from FFT analysis
dominant_freq_hz	<code>float</code>	The peak frequency found in 0.7–3.0 Hz band

SECTION 6: MASTER CHECKLISTS

Print This Page — Check Each Item Before Moving to the Next Phase

6.1 Pre-Coding Checklist [ALL MEMBERS — Phase 0]

✓ Phase 0 Checklist

- ☐ Git repository created and accessible by all 4 members
- ☐ Full folder skeleton pushed (all directories, all empty `__init__.py` files)
- ☐ `requirements.txt` created with: `opencv-python`, `numpy`, `scipy`, `matplotlib`, `imutils`
- ☐ All 4 members have run `pip install -r requirements.txt` successfully
- ☐ All 4 members have verified webcam works with `cv2.VideoCapture(0)`
- ☐ `config.py` exists with: `FPS=30`, `BUFFER_SIZE=300`, `ROI_PADDING=10`, `BANDPASS_LOW=0.7`, `BANDPASS_HIGH=3.0`, `THREAT_THRESHOLD=0.85`
- ☐ Module interface contracts written as comments in each `__init__.py`
- ☐ Team communication channel active and all 4 members are in it
- ☐ Haarcascade XML file confirmed available (Person A)
- ☐ At least 1 test video per member available locally

6.2 Person A Checklist [Face Detection + Integration]

✓ Person A Checklist

- ☐ `face_detector.py`: detects face in a static JPEG image and returns bounding box
- ☐ `face_detector.py`: handles frames with NO face gracefully (returns `face_detected=False`)
- ☐ `face_detector.py`: returns the LARGEST face if multiple are present
- ☐ `roi_extractor.py`: extracts forehead sub-region (top 30% of bounding box, center 60% width)
- ☐ `roi_extractor.py`: extracts cheek sub-region (middle 40% height, outer thirds)
- ☐ `roi_extractor.py`: returned arrays have shape (h, w, 3) and are not empty
- ☐ `DataPacket` dictionary schema published to team (all 4 members acknowledge)
- ☐ `main.py` skeleton written with video capture loop and placeholder calls to all 4 modules
- ☐ `main.py`: pressing Q cleanly exits the `cv2` loop and releases the capture
- ☐ Integration test 1: full pipeline on YOUR OWN FACE runs without exception

- ☐ Integration test 2: full pipeline on a video FILE runs without exception
- ☐ Face tracking fallback implemented (last bbox reused if face lost for 1 frame)

6.3 Person B Checklist [rPPG Signal Engine]

✓ Person B Checklist

- ☐ signal_extractor.py: correctly extracts green channel mean from a single ROI array
- ☐ signal_extractor.py: accumulates values into a rolling circular buffer of BUFFER_SIZE length
- ☐ preprocessor.py: applies scipy.signal.detrend to remove linear drift
- ☐ preprocessor.py: output signal is normalized to zero mean and unit variance
- ☐ preprocessor.py: handles edge case of buffer not yet full (returns partial array or zeros)
- ☐ rppg_algorithms.py: GREEN method implemented and returns a BPM estimate
- ☐ GREEN method tested with synthetic sine wave: BPM output is within ± 5 of input frequency $\times 60$
- ☐ CHROM or POS method implemented (at least skeleton — even if results are imperfect)
- ☐ is_calibrating flag correctly True for first (BUFFER_SIZE / FPS) seconds, then False
- ☐ Module has no import errors and runs standalone with random numpy input
- ☐ Output dictionary matches the agreed schema from Section 5.2
- ☐ Team informed of exact numpy array shape and dtype of rppg_signal output

6.4 Person C Checklist [Threat Analysis Engine]

✓ Person C Checklist

- ☐ fft_analyzer.py: correctly computes FFT and frequency bins for a given sample rate
- ☐ fft_analyzer.py: bandpass filter isolates only 0.7–3.0 Hz range
- ☐ fft_analyzer.py: returns dominant frequency and SNR metric
- ☐ fft_analyzer.py: tested on synthetic 1.2 Hz sine — dominant_freq is between 1.1–1.3 Hz
- ☐ pattern_detector.py: computes normalized autocorrelation of signal
- ☐ pattern_detector.py: synthetic perfect loop (repeated sine) scores loop_detected=True
- ☐ pattern_detector.py: real noisy signal scores loop_detected=False
- ☐ threat_scorer.py: inputs (pulse_present, loop_score, snr_score) → outputs verdict dict
- ☐ All three test cases pass: REAL signal → REAL, perfect loop → THREAT, flat signal → THREAT

- ☐ Rolling 5-frame majority vote implemented to prevent verdict flickering
- ☐ Thresholds defined in config.py (not hardcoded in the module)
- ☐ Output dictionary matches the agreed schema from Section 5.3

6.5 Person D Checklist [UI / Demo / Presentation]



Person D Checklist

- ☐ display.py: draws bounding box in correct color (green=REAL, red=THREAT, yellow=UNCERTAIN)
- ☐ display.py: displays verdict text in large font (cv2.putText, font scale ≥ 1.5) at top of frame
- ☐ display.py: displays BPM and confidence percentage as text overlay
- ☐ display.py: shows "CALIBRATING..." during warm-up period instead of a verdict
- ☐ plotter.py: scrolling rPPG signal graph visible on screen (either matplotlib window or cv2 overlay)
- ☐ logger.py: results.csv file is created and each verdict is appended correctly
- ☐ logger.py: CSV has headers: timestamp, verdict, confidence, bpm, loop_detected
- ☐ At least 2 real face video clips in /data/test_real/ (each ≥ 10 seconds)
- ☐ At least 2 deepfake video clips in /data/test_fake/ (each ≥ 10 seconds)
- ☐ docs/architecture.png created and committed (system block diagram)
- ☐ README.md complete: Prerequisites, Installation, Run command, Controls, Interpretation guide
- ☐ docs/demo_script.md written with 3-minute narrative
- ☐ Screen recording of successful deepfake detection saved as insurance for the demo

6.6 Final Integration Checklist [ALL MEMBERS — Phase 3+4]



Final Integration Checklist

- ☐ main.py imports all 4 modules without errors
- ☐ End-to-end pipeline runs on live webcam (no exceptions)
- ☐ End-to-end pipeline runs on /data/test_real/ video file
- ☐ End-to-end pipeline runs on /data/test_fake/ video file
- ☐ REAL face consistently scores REAL verdict ($\geq 80\%$ of frames after calibration)
- ☐ Deepfake video consistently scores THREAT verdict ($\geq 70\%$ of frames after calibration)
- ☐ Display overlay is clear and readable from 2 meters distance

- ☐ rPPG signal graph is visible and updates in real time
- ☐ FPS is acceptable: ≥ 10 FPS on demo machine (display FPS counter)
- ☐ results.csv is being written correctly
- ☐ All 4 members have done a final git pull and confirmed clean state
- ☐ System can be started with a single command: `python main.py`

SECTION 7: DEMO SCRIPT & JUDGE FAQ

How to Present, What to Say, How to Handle Hard Questions

7.1 3-Minute Demo Script (Person D leads, all members present)

 **Practice this at least twice before the presentation. Each step has a time budget.**

#	Time	What to Show / Say	Who
1	0:00–0:25	THE HOOK: "Every deepfake detection tool today looks at pixels. Ours looks at the heartbeat. A deepfake has perfect pixels but zero pulse. We exploit that gap."	Any member
2	0:25–0:45	THE SCIENCE: "This technique is called rPPG — Remote Photoplethysmography. Blood absorbs green light differently when it pumps. We measure that invisible flicker at 30fps."	Person B
3	0:45–1:15	DEMO — REAL FACE: Launch system on live webcam. Point at your own face. Show the GREEN box, the BPM reading (60–80), the scrolling heartbeat signal on screen.	Person D
4	1:15–1:50	DEMO — DEEPFAKE: Switch input to deepfake video file. Show the system flip to RED box and "⚠ THREAT DETECTED". Point at the flat rPPG signal.	Person D
5	1:50–2:15	THE ARCHITECTURE: Show the block diagram. "Four modules: face detection, signal extraction, threat analysis, visualization — all running in real time."	Person A
6	2:15–2:45	THE IMPACT: "This is physics-invariant. You cannot train a GAN to fake a heartbeat without adding a full cardiac simulation layer. This makes our defense future-proof."	Person C
7	2:45–3:00	THE CLOSE: "Biometric physics as cybersecurity. The body becomes the password. The pulse becomes the proof."	Any member

7.2 Anticipated Judge Questions & Prepared Answers

Q: "Can't you just train a GAN to generate a realistic rPPG signal?"

Prepared answer: Yes — theoretically. But it requires the generative model to (a) know it will be rPPG-analyzed, (b) have a cardiac simulation layer producing realistic 1/f noise, and (c) synchronize that signal with the rendering in the exact frequency domain. Current deepfake pipelines (FaceSwap, FaceForensics, Stable Diffusion video) do not do this. The signal would need to be biologically realistic — not just a looped sine wave. This raises the attack cost enormously and makes our defense time-asymmetric in our favor.

Q: "What is your false positive / false negative rate?"

Prepared answer: Our prototype is optimized for demo conditions. In controlled lighting, we achieve consistent REAL detection on live faces and THREAT detection on rendered deepfakes. In a production system, we would tune the threshold set using labeled datasets (FaceForensics++ provides ground truth) and report an AUC-ROC curve. Our current threshold is conservative: we prefer a false alarm over a missed detection for security-critical applications.

Q: "What about someone moving a lot or poor lighting?"

Prepared answer: Motion and lighting are the two primary noise sources in rPPG. We handle motion with ROI stabilization (tracking the face across frames) and lighting with linear detrending of the raw signal. In production, we would add ICA (Independent Component Analysis) to separate the blood volume pulse component from motion artifacts — this is the standard academic approach used in clinical rPPG research.

Q: "What happens with dark skin tones?"

Prepared answer: The melanin content in darker skin absorbs more light overall, reducing signal amplitude. However, the relative change due to blood volume pulse still occurs. We would tune our bandpass filter cutoffs and potentially switch from the green channel to the red channel (which penetrates deeper into skin tissue). Academic studies confirm rPPG works across skin tones with appropriate calibration.

Q: "Is this actually deployable in the real world?"

Prepared answer: Yes — this is a well-established research area. Medical-grade rPPG is used in commercial contact-free vital sign monitors. For deepfake detection specifically, companies like Sensity AI and Reality Defender are already building biometric-based detection. Our prototype demonstrates the core proof of concept; productionizing would involve GPU acceleration, multi-face tracking, and a cloud API layer.

SECTION 8: CONTINGENCY PLANS

What to Do When Things Go Wrong — And They Will

 **Read this section before you start. Knowing your fallback options means you won't panic when a bug appears at Hour 3.**

8.1 If the rPPG Signal Is Flat / Noisy (Person B emergency)

- First check: is the ROI actually showing skin? Print `roi_forehead.shape` and `roi_forehead[:5,:5]` to confirm non-zero pixel values
- Second check: is the green channel being extracted correctly? Print the mean green value per frame — should vary between roughly 80–200 for skin
- If signal is noisy: increase ROI size (lower `ROI_PADDING` in `config.py`). A larger skin area averages out more noise.
- If detrending destroys the signal: skip detrending, use only normalization
- Fallback algorithm: use simple GREEN channel only (no CHROM/POS). Less accurate but more robust.
- Nuclear option: hardcode a SYNTHETIC test signal in B's module and demonstrate the detection pipeline works on that — explain to judges that real rPPG requires better camera hardware

8.2 If Face Detection Fails (Person A emergency)

- Check lighting: face detection requires reasonable illumination. Turn on a desk lamp or move closer to a window.
- Tune `scaleFactor` to 1.05 (slower but more sensitive) and `minNeighbors` to 3 (allows more detections)
- If Haar cascade fails entirely: switch to OpenCV DNN face detector (uses a ResNet-based model, more accurate). Download from OpenCV face detection GitHub page.
- Ultimate fallback: manually hardcode the ROI as the center third of the frame. This works for a demo where the subject sits still in front of the camera.

8.3 If Integration Breaks in Phase 3 (Person A emergency)

- Isolate the broken module: test each module individually by calling it with its stub input. Find which module is returning the wrong type.
- The most common integration bug: numpy array dtype mismatch (float64 vs uint8). Always convert with `.astype(np.float64)` when entering the rPPG pipeline.
- Second most common: shape mismatch. Print `.shape` at every module boundary to trace the error.
- If `main.py` cannot be fixed in 20 minutes: run modules independently in separate terminals and demo them side-by-side. Less impressive but still shows the science.

8.4 If Deepfake Videos Are Not Available (Person D emergency)

- Source 1: FaceForensics++ on YouTube — search "FaceForensics deepfake sample" for publicly shared examples
- Source 2: The DeepFake Detection Challenge (DFDC) has public preview samples on Kaggle

- Source 3: Play a deepfake YouTube video on a second laptop/phone and point the detection camera at the screen — the recorded deepfake on a screen still has no real rPPG signal
- Emergency option: generate a synthetic "fake" signal in Person C's module and demo the threat detection without needing a real deepfake video. The math still holds.

8.5 Minimum Viable Demo (if everything goes wrong)



If you reach Hour 4 with a broken system, switch to this fallback immediately rather than continuing to chase bugs.

- Display a working rPPG signal extraction on a real face with BPM displayed — even without threat detection, this demonstrates the core technology
- Show the FFT frequency spectrum in a matplotlib plot next to a real face video vs a flat/synthetic signal — the visual difference is compelling
- Show the architecture diagram and walk judges through the algorithm — a clear conceptual explanation is worth more than a crashing demo
- Play the pre-recorded screen capture of an earlier working state — always have this saved

SECTION 9: LIBRARIES, TOOLS & REFERENCES

What to Install, Where to Look, What to Read

9.1 Python Dependencies — requirements.txt

Package	Used By	Purpose
<code>opencv-python</code>	A, D	Video capture, face detection, frame display, image drawing
<code>numpy</code>	A, B, C	Array operations, FFT, signal math — the foundation of everything
<code>scipy</code>	B, C	signal.detrend, signal.butter bandpass filter, signal.correlate
<code>matplotlib</code>	B, D	Real-time signal plotting, figure-to-image conversion
<code>imutils</code>	A	Convenience functions for OpenCV: resize, grab_contours, FPS counter
<code>Pillow (optional)</code>	D	Image manipulation if matplotlib backend has issues
<code>scikit-learn (optional)</code>	C	PCA / ICA for advanced signal separation (if time permits)

9.2 Key Academic References (for judge Q&A)

- de Haan G. & Jeanne V. (2013) — "Robust pulse rate from chrominance-based rPPG" — IEEE Transactions on Biomedical Engineering. This is the CHROM algorithm.
- Wang W. et al. (2017) — "Algorithmic principles of remote PPG" — IEEE Transactions on Biomedical Engineering. This is the POS algorithm.
- Li X. et al. (2014) — "Remote heart rate measurement from face videos under realistic situations" — CVPR. Applied rPPG to challenging conditions.
- Chingovska I. et al. (2022) — "Anti-spoofing using rPPG analysis" — demonstrates rPPG for liveness detection, closely related to deepfake detection.

9.3 Useful Online Resources

- OpenCV Haarcascade files: github.com/opencv/opencv/tree/master/data/haarcascades
- FaceForensics++ dataset and papers: github.com/ondyari/FaceForensics
- Scipy bandpass filter tutorial: scipy.org → signal → butter → sosfilt
- Real-time matplotlib in OpenCV: search "matplotlib to opencv numpy tostring_rgb"
- rPPG Python implementations (reference only): github.com/habom2310/Heart-rate-measurement-using-camera
- draw.io for architecture diagram: app.diagrams.net (free, browser-based)

9.4 config.py — All Constants Reference

Constant	Default Value	What to change if...
----------	---------------	----------------------

FPS	30	Your webcam runs at 60fps → set 60; 720p webcam may be capped at 25
BUFFER_SIZE	300	Reduce to 150 if latency is too high; increase to 450 for better frequency resolution
ROI_PADDING	10	Increase if ROI is cutting off skin; decrease if including non-skin areas
BANDPASS_LOW	0.7	Lower (0.5) if subject has a very slow heart rate; keep > 0.5
BANDPASS_HIGH	3.0	Raise (4.0) for very athletic/stressed subjects; lower (2.5) to reduce noise
THREAT_THRESHOLD	0.85	Lower (0.70) if deepfakes are not being caught; raise (0.92) if real faces are false-alarming
LOOP_CORR_THRESHOLD	0.92	Lower to catch more loops; raise if real signals with repetitive motion are false-flagging
SNR_THRESHOLD	5.0	Signal quality threshold; lower for poor lighting conditions
MIN_FACE_SIZE	(80, 80)	Increase if small faces are false-detected; decrease if face not found on large screen
VERDICT_WINDOW	5	Number of frames to vote across; increase for stability, decrease for faster response

SECTION 10: TEAM COMMUNICATION PROTOCOL

How to Stay Aligned Without Blocking Each Other

10.1 Sync Points — Mandatory Check-ins

Time	Type	What to share
T + 0:15	KICKOFF sync (5 min)	Confirm: everyone cloned, pip installed, webcam works. Assign tasks.
T + 1:00	Status ping (2 min)	Each person states: "I have X done, I'm blocked on Y". Text-only is fine.
T + 1:30	Interface review (5 min)	B shows C their rppg_signal output format. A shows everyone DataPacket.
T + 2:30	Mid-point check (5 min)	Everyone demos their standalone module. Identify blockers for integration.
T + 2:45	Integration kickoff (5 min)	A pulls everyone's latest code, starts wiring main.py. Others go on-call.
T + 3:30	Integration complete (5 min)	Full pipeline demo on A's screen. Identify remaining bugs.
T + 4:15	Demo rehearsal (10 min)	Full 3-minute demo run-through. Time it. Fix presentation issues.

10.2 Communication Rules

- If you finish a task EARLY: help Person D with test data sourcing or write unit tests — do not start new features
- If you are BLOCKED for > 20 minutes: immediately announce in the team channel. Do not silently struggle.
- If a MERGE CONFLICT appears: the person whose code is conflicting resolves it. Never force-push. Never rebase.
- If you need to modify config.py: announce the constant name and value in team chat before committing. Wait 30 seconds for objections.
- If Person A is deep in main.py integration: do NOT push new changes to your module without warning. Announce first.
- Celebrate small wins in the team chat — it keeps energy high during a sprint. "Signal extraction working! 🎉" matters.

 **FINAL REMINDER:** The goal is not perfect code. The goal is a compelling, working demonstration of a paradigm-shifting idea. A demo that shows a REAL face with a heartbeat and a DEEPFAKE with no heartbeat — detected in real time — is a winner. Ship it.